Clone simple steps, then fix the view

Research report, 25 October 2026. Audience: two beginner master's students (team M, Project 6, Niryo Ned2 imitation learning, submission 10 January 2027). Every number carries its source. Statements that are our own reasoning rather than a published finding are marked "(inference)". Results from 2016 to 2022 are marked "(older)" where that matters.

Executive summary (plain language)

Train a small convolutional neural network (CNN, an image-recognition network) that looks at one wrist-camera picture and picks the next 1 cm step from a short list of moves. This is plain behaviour cloning (BC): supervised learning from your own demonstrations. It is the best-evidenced method for your data size (100 to 300 episodes), your laptop, and your one-step-per-second robot.

Do not start with the large 2023 architectures ACT (Action Chunking with Transformers) or Diffusion Policy. Their measured gains come from fast 10 to 50 Hz control and from predicting many future actions at once. You decide once per second and can see only about 7 cm ahead, so those gains do not apply. Keep ACT as an optional upgrade if a university GPU appears.

Collect demonstrations by stepping the robot with the keyboard through the exact capture-then-move loop the trained model will later use. Hand-guiding in learning mode puts your hand into the camera view and gives blurred frames. A scripted teacher does not match the professor's wording and needs his approval even as a helper. Aim for 200 to 300 episodes spread over 20 to 30 printed paths, not 50 repeats on each of six sheets; variety of local line shapes matters more than repeats. Write one stepping rule and both follow it.

After the first model works on seen sheets, run two or three correction rounds: the model proposes a step, you accept or override with a key, and only your overrides become new labels. This "human-gated" correction is the single most reliable fix for drift in every real-robot study found. Always use random-crop image augmentation; it is the one training trick with hard numbers behind it.

The biggest risk is not the learning method but the camera. With the pusher vertical it sees the line only when the path heads within about 45 degrees of "away from the base". Hairpins, meanders and loops contain stretches it cannot see at all. Measure the visible fraction of each sheet this week and ask the professor two questions now: what the unseen test sheet will contain, and whether the camera may be mounted on the hand so it turns with the pusher. Evaluate on the robot, never by validation accuracy: pre-register a success rule, run 20 trials per sheet, report counts with intervals, progress along the line, and failure types.

1. The state of the art favours a small, discrete-step behaviour-cloning policy

Plain behaviour cloning already saturates on easy, unambiguous tasks at your data scale

The most thorough study of learning from small human datasets is still robomimic (2021, older). With **200 demonstrations** per task and image input, plain BC with a ResNet-18 encoder reached **100 percent on Lift and 97.3 percent on Can**, but only 62.0 percent on Square and 55.3 percent on Transport; a recurrent variant (BC-RNN, a long short-term memory network over 10 frames) lifted those to 82.0 and 72.0 percent (robomimic). The dataset-size ablation is the number that matters for you: with only **20 percent of the data, about 40 demonstrations, Lift still scored 100 percent and Can 83.3 percent**, while Square and Transport collapsed to about 30 percent (robomimic). Your task, keeping a high-contrast black line centred under a red puck that sits at the bottom of every frame, is closer to Lift or Can in decision complexity than to Square (inference). The same paper gives the design rules that transfer directly: learning rate 1e-4 (1e-3 dropped image agents by 35 to 63 percent), a ResNet-18-sized encoder (a shallow encoder dropped performance 25 to 62 percent), no oversized MLP head (larger heads overfit), and a Gaussian mixture model (GMM) head rather than a deterministic one on multi-operator data (robomimic).

Pretrained vision encoders are not a free win. Hansen et al. (ICML 2023) showed that a **5-layer convolutional network trained from scratch with random-shift augmentation is competitive with frozen pretrained encoders** (R3M, MVP, PVR): on Adroit Pen, from-scratch BC went from 22.0 to 74.8 percent with augmentation versus 81.3 for the best frozen encoder, and frozen pretrained features only did "marginally better" at about 10 demonstrations (Hansen et al.). The 2025 DINOv3 comparison found that a **frozen DINOv3 encoder scored worse than a ResNet-18 trained from scratch** on Push-T, Can and Square (0.39 versus 0.78 on Push-T) and only caught up when fine-tuned (DINOv3-Diffusion Policy). VC-1 reports that among pretrained visual representations "none are universally dominant" (VC-1). Our reading: an ImageNet-pretrained ResNet-18, fine-tuned with augmentation, or Hansen's 5-layer network, are the two low-risk encoders; frozen vision-transformer features are not worth the extra forward pass on a CPU (inference).

The closest analogues are visual line and lane followers, and they used tiny networks. PilotNet (2016, older) steered a car from a single camera with **about 250 thousand parameters** (Bojarski et al.). The 2019 AI Driving Olympics lane-following winner used a **64x32 pixel input and a 5-convolution network that runs on one CPU**, trained on 20,294 images (Sazanovich et al. 2020). The Learning-to-Drive benchmark's imitation baseline used 40x40 grayscale frames and a categorical (classification) head (Viitala et al. 2020). DonkeyCar's documentation calls its categorical steering model "very robust" on limited compute (DonkeyCar Keras docs). A three-class (straight, left, right) CNN path follower completed its course in all 40 trials while reaching only 82 percent classification accuracy, and its authors noted that numerical accuracy "was not the best indicator" of closed-loop success (West Point report, search snippet only).

ACT and Diffusion Policy solve problems you do not have at one step per second

ACT trains an **80 million parameter** transformer with a conditional variational autoencoder on 50 demonstrations per task, predicts chunks of **100 actions at 50 Hz**, and needs about **5 hours on an 11 GB RTX 2080 Ti** (ACT). Its ablation shows that without chunking (chunk size 1) success is about 1 percent, rising to about 44 percent at chunk size 100 (ACT). Diffusion Policy used **136 real demonstrations** for Push-T, a fixed **12-hour training budget**, and about 0.1 s inference on an RTX 3080 with 10 DDIM steps (Diffusion Policy); a later measurement puts a transformer diffusion policy at **5.2 seconds per inference on CPU** versus 207 ms for VQ-BeT (VQ-BeT).

A 2026 analysis explains why chunking helps and when it does not. On Libero-90 a single-step Markovian diffusion policy got 68.9 percent, chunking 89.2, and a history-conditioned single-action policy 94.0; the authors conclude "the temporal consistency of action chunking does matter, but only at high frequencies" (50 to 60 Hz), and attribute the remaining gain to conditioning on past observations and implicit ensembling (Why chunking helps, 2026). Bidirectional Decoding adds that "there is no universally optimal choice of action horizon" and that the best chunk length shrinks to 1 as environment noise rises (Bidirectional Decoding). Translated to your robot (inference): an ACT chunk of 100 would mean 100 seconds of blind motion; Diffusion Policy's 8-step execution horizon at 10 Hz is 0.8 seconds, which is one of your steps. A chunk of 2 or 3 would push the puck 2 to 3 cm without looking while only about 7 cm of line is visible, which is risky at a hairpin. ACT remains feasible through LeRobot if a GPU is available (LeRobot calls it "the first model we recommend when you're starting out" and expects "a few hours for 100k training steps on a single GPU" (LeRobot ACT docs)), but its machinery is not what will decide your result.

Implicit, nearest-neighbour and tokenised policies help only with multimodal data

Implicit BC (2021, older) beats explicit mean-squared-error BC when several actions are valid for one image, but trains about **5 times slower** (17.9 versus 101.3 steps per second), needs gradient penalties and spectral normalisation, and scored **0 percent on real Push-T** against Diffusion Policy's 95 (IBC; Diffusion Policy). BeT and VQ-BeT discretise actions with k-means or vector quantisation and ran on 45 to 1,000 demonstrations, with their advantage smallest on the largest dataset and no reduced-data experiment (BeT; VQ-BeT). VINN (2021, older) replaces training with nearest-neighbour lookup over pretrained embeddings and won on a real door task with 71 demonstrations (**80 percent versus 0 for end-to-end BC**), with its edge concentrated below about 20 demonstrations (VINN). A line-following push has little multimodality: given the picture there is normally one correct step, except at self-crossings and at the moment a hairpin enters view (inference). A classification head already represents a two-way ambiguity as two probable classes instead of averaging them into a diagonal step. VINN is worth an afternoon as a no-training diagnostic baseline; Implicit BC, BeT and VQ-BeT are not justified here (inference).

Discrete 1 cm steps are the theoretically favoured action space

Two recent theory papers back the discrete-step design. Simchowitz et al. (COLT 2025) prove that "any smooth, deterministic imitator policy necessarily suffers error on execution that is exponentially larger" than its training error, and that escaping this requires non-smooth, non-Markovian or state-dependently stochastic policies (Simchowitz et al.). Cao and Xie (ICML 2026) show that **BC with quantised actions and log-loss achieves optimal sample complexity** with only polynomial dependence on horizon under stable dynamics (Cao and Xie). Your demonstrations are produced one discrete step at a time, so there is no quantisation error at all (inference). The empirical side agrees: robomimic's deterministic head lost up to 58 percent against a mixture head on multi-operator data (robomimic), and every real-hardware learned pusher used an end-effector position target, absolute or relative, rather than joint targets or velocities (Diffusion Policy; Wei et al. 2025; Zhang et al. 2024). Relative steps are also forced by your sensor: the absolute position on the sheet is not observable from a wrist camera that always shows the puck at the bottom (inference).

The class list should mirror what the demonstrators actually do (inference): forward, forward-left, forward-right, left, right, backward, one or two "reposition" moves (lift and step around the puck) if hairpins are attempted, and stop. Keep diagonal classes if you use them, or the puck will zig-zag on curves. The stop class needs its own positive examples: adversarial imitation work (2020, older) pads every demonstration with an explicit "Done" action (Done action), and Diffusion Policy warns that idle actions left in the data made baselines "get stuck" (Diffusion Policy), so log exactly one

frame per issued move and tie the stop label to a visible end marker, not to a pause.

Image input: random crop is the one non-negotiable

Random-crop ("pixel shift") augmentation is the most repeatedly measured design choice in this literature. Removing it caused **relative drops of 47 percent (Square) and 35 percent (Transport)** in simulation and dropped the **real Can task from 73.3 to 26.7 percent** (robomimic); it more than tripled from-scratch BC success in Hansen et al. (22.0 to 74.8 on Adroit Pen) (Hansen et al.); and standard augmentation raised real pushing BC from 62.5 to 87 percent on unseen objects (Young et al. 2020). Standard practice is 84x84 to 120x120 inputs cropped to about 90 percent of the side (robomimic; Diffusion Policy). A 640x480 frame downscaled to about 160x120 and randomly cropped to about 144x108 keeps the 1 cm line at roughly 2 to 3 pixels wide at the top of the frame given the 9 cm field of view; below about 96x72 the far line risks vanishing (inference, from the stated field of view). Colour jitter is the standard second augmentation, but LeRobot's advice is to "start conservative" with brightness 0.9 to 1.1 (LeRobot dataset v3 docs); strong hue or saturation jitter could erase the red-puck and black-line cues the policy depends on (inference). Start with a single frame: Diffusion Policy found an observation horizon of 2 best for vision and performance decreasing beyond it (Diffusion Policy), and histories invite the copycat shortcut discussed in section 3.

Method	Demos used in source	Training compute in source	Inference	Fit for this task
Plain BC, small CNN or ResNet-18, discrete head	40 to 200 (robomimic)	Minutes to an hour on a laptop CPU (inference)	Tens of ms CPU (inference)	Baseline
BC-RNN (10-frame GRU/LSTM)	200 (robomimic)	Slightly more than BC	Same	Fallback for hairpins, with copycat mitigation
VINN (nearest neighbour)	71 (VINN)	None if encoder frozen	About 0.11 s	Diagnostic baseline only
ACT	50 (ACT)	About 5 h, RTX 2080 Ti	About 10 ms GPU	Optional upgrade if GPU available
Diffusion Policy	136 to 200 (Diffusion Policy)	12 h budget, GPU	0.1 s GPU, about 5 s CPU (VQ-BeT)	Not justified
Implicit BC	50 to 500 (IBC)	About 5x MSE-BC, unstable	7 ms	Ruled out
BeT / VQ-BeT	45 to 1,000 (VQ-BeT)	Under an hour (BeT)	18 ms GPU, 207 ms CPU	Not justified (unimodal task)

No published result trains any of these at about 1 Hz with step-wise blocking demonstrations, and nobody has published BC through pyniryo on a Ned2; the mapping from the cited numbers to your regime is inference throughout.

2. Keyboard stepping plus human-gated corrections beats hand-guiding and scripted teachers

Keyboard stepping gives an exact match between training and test images

The only direct head-to-head on real manipulation (Li, Cui and Sadigh, 2025, Franka, 100 trajectories per modality per task, diffusion policy) found that **kinesthetic (hand-guided) demonstrations produced the best models on two of three tasks**, but only because the recorded poses were replayed by the robot to re-capture images "unimpeded by the demonstrator's hand"; the replay roughly doubled collection time on the contact-heavy task, and mixing kinesthetic with teleoperated data beat either alone by about 20 percent (Li et al. 2025). Earlier work states plainly that kinesthetic teaching "introduces visual distraction from the human's presence in the scene" (force-feedback demonstrations, 2022). Because pyniryo moves block, a replay of a hand-guided path would itself be a sequence of 1 cm step moves, so hand-guiding collapses into keyboard stepping plus extra time (inference). Keyboard stepping through the same capture-move loop that the deployed policy will use means the training images are literally the test-time images: static arm, no motion blur, no hand (inference). The usual penalty on keyboard teleoperation, jerky axis-aligned motion (Isaac Lab docs), does not apply because the action space is already a 1 cm step and the arm settles between steps (inference).

What does matter is consistency. Belkhale, Cui and Sadigh (NeurIPS 2023) found that on robomimic Square, BC success fell from **58 percent (proficient) to 36, 12 and 2 percent** as operator quality fell, that "state diversity was not correlated with performance", and recommend curating "toward more consistent actions" (Belkhale et al.). robomimic's multi-human data (six operators) lagged its single proficient operator on the harder tasks (Transport 42.0 versus 72.0 for BC-RNN) and the authors blame "suboptimal and multimodal data" (robomimic). Two students who follow one written stepping rule (for example: step along the line's local direction; if the puck centre is more than half a line width off, take one diagonal step toward the line; at a crossing continue straight; at a hairpin perform the same fixed repositioning sequence) will look like one proficient operator rather than two inconsistent ones (inference).

A scripted teacher that knows the line coordinates is powerful but conflicts with the brief. Learning by Cheating (2019, older, driving simulation) turned a **20 percent camera-only policy into a 100 percent one** by letting a privileged teacher label every state the student reached (Learning by Cheating). A 2025 paper warns that querying such a teacher at states far from anything realisable makes "a dataset that becomes increasingly unrealizable, causing policy divergence" (critical-state querying). The professor's words are "we push the puck ourselves using the robot"; a scripted teacher is not that. Present it to him, if at all, as an optional labelling aid for recovery steps near the line, and do not use it as the primary source without his approval (inference).

Human-gated corrections are the most reliable fix for drift

Every real-robot study that compared plain BC with on-policy corrections found BC failing at rarely visited states and corrections fixing it. On the AutoRally dirt track (2018, older), batch imitation with 6,000 to 12,000 samples completed **51 to 69 percent of runs** and crashed in "an area rarely explored in the expert's demonstrations", while online DAgger (Dataset Aggregation) completed **100 percent** in every case (Pan et al. 2018). On a real test vehicle, HG-DAgger (Human-Gated DAgger, 2019, older) recorded **0 collisions and 0 road departures in the first 5,000 samples against 11 and 66 for BC**, starting from 10,000 BC labels and adding five rounds of 2,000 (Kelly et al.). In HG-DAgger the human takes over only when the policy enters an unsafe region, and "expert action labels are only collected and added to D during these recovery trajectories"; the authors argue this yields higher-quality labels than classic DAgger's shared control (Kelly et al.). For eye-in-hand pushing specifically, synthesised recovery views raised BC from **30 to 100 percent with 24 demonstrations** and from 20 to 80 with 8 (Zhang et al. 2024). RaC (2025) reports about 10 times less collection time than prior methods with a rewind-then-correct protocol (RaC), and LeRobot now ships this loop as `lerobot-rollout --strategy.type=dagger` with the guidance to intervene on "irreversible mistake", "hesitating" or "deviating", and to keep corrections "confident, clean" (LeRobot HIL docs).

The cheapest version for a blocking 1 cm interface (inference): the collection script shows the policy's proposed step on screen; the operator presses accept (the robot executes the policy step, frame stored as "policy") or presses the correct direction (the robot executes the human step, frame stored with the human label). Train on demonstrations plus human-labelled corrections only. Because each step is 1 cm and blocking, the actuator-lag and safety problems that motivated gating in driving do not arise. Scaling HG-DAgger's schedule to your budget suggests **100 to 150 initial episodes, then 20 to 40 correction episodes per round for 2 to 3 rounds**. The human still pushes the puck using the robot in every correction and every label is human, so this stays inside "imitation learning only", but the autonomous segments should be described openly to the professor (inference).

DART (2017, older) offers an off-policy complement: inject small noise into the demonstrator's commands so the data contains recoveries, with the clean intended action as the label; it gave a 62 percent average improvement over BC on physical grasping in clutter (DART; BAIR blog). Belkhale et al. found that transition noise during collection helped (Square 82.0 percent at sigma 0.2 versus 55.7 at 0.05) while noise on the expert's labels "often hurt, especially with small datasets" (Belkhale et al.). A discrete analogue (inference, magnitude is a guess to tune on the first 20 episodes): with probability 0.15 to 0.25 per step the script executes a random lateral step instead of the key pressed, the operator then corrects, and the perturbed frame is labelled with the operator's intended clean step. Physically starting a fraction of episodes with the puck offset 0.5 to 1 cm or the tool 15 to 30 degrees off the tangent achieves the same thing with no code (inference).

Two hundred to three hundred episodes are enough if they are varied, not repeated

The best 2025 evidence on how many demonstrations are needed is the data-scaling-laws study with over 40,000 UMI (Universal Manipulation Interface) demonstrations: generalisation follows a power law in the number of distinct environments and objects, performance per environment-object pair **plateaus at about 50 demonstrations**, "adding environments helps even when total demonstrations are held constant", and 32 pairs times 50 demonstrations gave 85 to 92.5 percent success in eight unseen environments (Lin et al. 2025). LeRobot's default is "at least 50 episodes, with 10 episodes per location", with the warning to "avoid adding too much variation too quickly" (LeRobot il_robots). For pushing specifically, real-only policies on a Push-T-style task scored **2 of 20 with 10 demonstrations, 10 of 20 with 50, and 20 of 20 with 150** (Wei et al. 2025). Line followers worked from similar frame counts: DonkeyCar's guide targets about 10,000 records (DonkeyCar docs), the AI Driving Olympics winner used 20,294 training images (Sazanovich et al. 2020), and the Learning-to-Drive imitation baseline used 10,000 steps (Viitala et al. 2020). Your 100 to 300 episodes of 25 to 40 steps give **2,500 to 12,000 labelled frames**, the same order as all of these (inference).

For your wrist camera the "environment" of the scaling law is not the whole sheet but the local line geometry inside the 9x7 cm window: straight, gentle arc, tight arc, hairpin entry, crossing, line end, each at various puck offsets (inference). Six sheets already contain these shapes, but printing 20 to 30 extra sheets is cheap and lets you vary curvature radii and crossing angles so that the unseen test sheet's local shapes fall inside the training distribution (inference). The Learning-to-Drive result is the warning: agents trained on the mixed track transferred at 100 percent, while agents

trained only on the sharp-right-angle track scored 21 to 25 percent elsewhere (Viitala et al. 2020). Spend the budget as roughly 10 to 15 episodes per printed path, starting at several points along each line, in both directions where the camera allows, with deliberate lateral offsets; PilotNet oversampled curve frames to counter straight-road bias (Bojarski et al.), and you should weight hairpin and tight-arc frames the same way (inference). Keep lighting stable in the first dataset (HF dataset checklist) and add moderate variation only after the policy works on seen sheets.

Quality filtering and logging hygiene

Discard failed episodes and episodes with reversed steps, but keep imperfect successes: robomimic's BC-RNN did better on all multi-human data than on the best operators only (Square 78.0 versus 66.0) (robomimic), and the AI Driving Olympics 2020 entry deliberately kept "imperfect trajectories" to teach recovery (Sazanovich et al. 2020). Isaac Lab's rules translate directly: keep demonstrations short, take a direct path, do not pause (Isaac Lab docs). Learned weighting schemes such as DemInf gave 5 to 10 percent on much larger sets (DemInf) and are not worth implementing at your scale (inference).

The blocking interface removes the hardest logging pitfall, observation-action timing mismatch, which UMI showed costs 30 points of success when ignored (87.5 to 57.5 percent) (UMI), but only if each image is captured after the previous move has returned and before the next is sent. The 9 fps JPEG stream will otherwise hand back a frame from during the motion; LeRobot's camera API documents that a latest-frame read "may be stale" (LeRobot cameras). The loop should therefore be: move returns, short settle, drain the stream and keep the newest frame timestamped after move completion, record frame and robot state, choose the action, log the action against that frame, send the move (inference). Store per step the episode id, step index, capture and command timestamps, joint angles and tool pose before and after, the action class, who chose it (human, policy, perturbation) and a done flag; per episode the sheet id, start index, initial offset, operator, date, lighting note, success flag and collection round (inference). Writing the same data in LeRobotDataset v3 format (Parquet tables plus MP4 frames with `meta/info.json`, `meta/episodes/`, and a mandatory `dataset.finalize()`) lets you use the dataset visualiser and `lerobot-train` without conversion (LeRobot dataset v3).

Augmentation that substitutes for real demonstrations

Beyond random crop and mild jitter, two older tricks map almost one-to-one onto your camera. PilotNet synthesised off-centre views by shifting and rotating the image and adjusted each label "to one that would steer the vehicle back to the desired location and orientation in two seconds" (Bojarski et al.); this idea dates to ALVINN in 1989 (NVIDIA PilotNet experiments). Shifting your frame by k pixels is what the camera would see with the puck k pixels off the line (after a one-off ruler calibration), relabelled as one step toward the line (inference); physical offsets are safer because synthetic shifts do not show the true puck contact geometry (inference). Mirror flipping with swapped left and right labels is supported by MirrorDuo (2026) when demonstrations cover both sides of the workspace (MirrorDuo), but one 2024 driving check found flips "did not improve its results" (BC reality check), and the flip is valid only if the camera is centred on the pusher axis; test it on held-out demonstrations before relying on it (inference).

3. The fixed forward camera, not the learning method, decides hairpins and loops

What the camera cannot see, no policy can follow

The team's own measurements fix the geometry: with the pusher vertical the camera always looks away from the base, sees a window about 9 cm wide and 7 cm ahead of the puck, and sees the line ahead only while the path heads within about 45 degrees of "away from base". Yaw by forearm roll works only near one spot about 20 cm from the base, and a hand-mounted camera would turn with the pusher but is limited to plus or minus 145 degrees. The consequence (inference) is severe: to push the puck toward the base, the rod must be on the far side of the puck, and the camera then looks away from both the puck and the upcoming line, so neither is in view. Any segment heading more than about 45 degrees off the "away" direction is not merely hard to see; it is unobservable. A full loop covers every heading, and a classic meander alternates headings, so those sheets contain stretches where the policy would be blind for 10 or more consecutive steps.

The literature confirms that this is fatal for a hand-only policy. Hsu et al. (ICLR 2022) found that eye-in-hand policies beat third-person ones when the target stays in view (69.6 versus 55.6 percent mean on Meta-World, 52 versus 20 percent under real shifts), but "when hand-centric observability is low, [the hand-only policy] both trains and generalizes worse" on tasks where the target leaves the hand view (Hsu et al.). The one study that quantified field-of-view failures in reactive visual control attributed **29.3 percent of all failures** to a too-narrow field of view, with the typical pattern "as it turns a new tree comes into view", and concluded "this problem inevitably afflicts purely reactive controllers" (Ross et al. 2013, older). A feed-forward CNN that followed indoor lanes "succeeded only occasionally" on a U-turn needing history

(NTNU thesis 2018). Mobile line followers handle line loss with hand-coded search behaviours rather than learned ones: the AI Driving Olympics 2020 entry "hard-coded the robot's motion" at standard turns and intersections (Sazanovich et al. 2020), and a 2026 camera-only recovery scheme turns in place, relaxes thresholds and returns to visual-odometry breadcrumbs, succeeding in 86.6 percent of 119 fault-injected simulated episodes (Self-healing recovery). Mobile robots are also easier than your arm in one respect: their camera turns with the vehicle, so a turn brings the line back; a fixed-heading wrist camera has no such mechanism (inference).

The first action is therefore measurement, not modelling (inference). For each of the six sheets, trace the line, compute the heading at each centimetre, and record the fraction of length within 45 degrees of "away from base" for the best sheet orientation. Sheets with 100 percent observable length (straight, most arcs, a gentle S) are in scope for the baseline. Sheets with long unobservable stretches are out of scope for any fixed-camera policy and must be discussed with the professor, because the unseen test sheet is drawn from the same family. There is also a discrepancy worth checking: a login-gated Niryo Academy course lists the wrist camera as a Sony IMX322 with about **104 degrees field of view** (Niryo Academy course 57, search excerpt), which at the measured lens height of about 8 cm would span roughly 20 cm (inference), twice the measured 9 cm window width. The robot stores its calibration, retrievable with `get_camera_intrinsics` (pyniryo tcp_client.py); a ruler test at the working height and a look at where the view is being cropped (by the pusher, by the camera height, or by a software crop) might reveal a cheap gain in visible area before any hardware change.

Memory bridges short gaps, not long blind stretches

Short histories are the standard default and longer naive stacks usually hurt. Diffusion Policy's vision variant "sees performance decrease as observation horizon increases", with 2 frames best (Diffusion Policy); Diffusion Transformer Policy and Dita report performance that "significantly drops" at a history of 3 (DiT Policy; Dita). The AI Driving Olympics winner tried multi-frame input and it "has not improved the performance" (Sazanovich et al. 2020). Recurrent policies do help where something must be remembered: robomimic's BC-RNN over 10 frames beat BC on every task (Square 82.0 versus 62.0, Tool Hang 67.3 versus 20.0) and sequence lengths of 10, 30 and 50 "did not differ significantly" (robomimic). Past-Token Prediction (2025) shows the sharpest picture: with a 16-step context, naive long-context diffusion collapses (Transport 0.00), no-history gets 0.60, and PTP with an auxiliary past-action loss gets 0.67, with real-world memory tasks at about 70 percent versus about 15 for no history, on 50 to 200 demonstrations (Torne et al. 2025).

The reason naive history fails is the copycat problem: in partially observed settings with autocorrelated expert actions, the imitator "learns to cheat by predicting the expert's previous action, rather than the next action", and "never switches actions when it should" (Wen et al. 2020, older). Your task is a textbook case (inference): "forward" dominates the data, the line leaves view, and the puck at the bottom of every frame makes the previous step partly recoverable from the image, so a naive history model would learn "repeat the last step" and push straight off the line at a hairpin. Any history design must either avoid history (current frame only), use a short GRU with the auxiliary past-action target of PTP, or at the very least never feed the previous action as an explicit input (inference). Image-based copycat mitigation is still described by its authors as "not significant yet" (Wen et al. 2020). At 1 cm per step, a 2-frame stack spans 2 cm and a 10-frame recurrent window spans 10 cm; memory can bridge a hairpin whose return leg is visible on approach and is only briefly hidden, but not a 10-step stretch that was never in view (inference).

Two cheaper tricks come from the driving analogues. Lookahead labels make the model commit to a turn while the curve is still visible: the DonkeyCar study found that predicting the label about 50 ms ahead gave the fastest safe laps (DonkeyCar speed study), and the eye-in-hand pushing paper labelled recovery frames with the pose at t+3 because t+1 "causes overshooting" (Zhang et al. 2024). Labelling each frame with the mode of the next 2 to 3 demonstrated steps is a one-line change (inference). Appending the last one to three commands as extra inputs, as Ross et al. did with a low-pass command history (Ross et al. 2013), is cheap but is exactly the copycat input, so test it only after the single-frame baseline and watch for "never turns" behaviour (inference).

A hairpin also needs a repositioning manoeuvre, which must be demonstrated consistently

Even with perfect vision a 2.4 cm puck on a 1 cm line cannot be pushed through a 180-degree turn from behind. Pushing an octagon along a path with a 90-degree turn "requires the robot to change the point of contact with the object during pushing" (Sieb et al. 2019), and the model-based pusher-slider controller handles direction changes by switching between sticking and sliding contact modes (Hogan et al. 2017, older). Rod pushing is also stochastic: over 2,000 identical 15 cm pushes the final pose scattered by 3.4 to 11.7 mm and 1.3 to 4.5 degrees, with a multimodal distribution (Yu et al. 2016, older). Scaled to 1 cm steps the per-step scatter is sub-millimetre, but a rod touching a disc off-centre makes the puck slide sideways, so the demonstrations must show the rod being re-centred behind the puck, and the action set must include sideways, backward and lift-and-step-around moves that the demonstrators actually use at hairpins (inference). If the manoeuvre is demonstrated as one fixed sequence, a classification head learns it without any

generative model; if it is demonstrated inconsistently, the modes get averaged and the policy hovers, which is the failure Diffusion Policy was built to avoid (Diffusion Policy) (inference). A separate "mode" class (push versus reposition), logged by a key press during collection, is a cheap interpretable addition with no evidence for or against it at your scale (inference).

Camera motion or a wider lens would convert the problem from memory to perception

Published active-vision work shows camera motion can be learned purely by imitation as extra action dimensions. AV-ALOHA appended 7 camera-arm joints to the ACT action vector, trained on 50 demonstrations per task, and beat wrist-only on the hidden-target tasks (Hidden Pick 55 to 60 percent versus 35), but "for those cases where AV is not necessarily advantageous, the added complexity from variation in viewpoint can negatively impact performance" (AV-ALOHA). Vision in Action put the neck pose in a 23-dimensional action and improved average task performance by 45 percent over wrist and fixed cameras, noting that wrist viewpoints "are constrained by manipulation requirements rather than guided by perceptual objectives" (Vision in Action). The wider-lens route has the cleanest numbers: a 2026 study found a single wrist fisheye (235 degrees in simulation) roughly **doubled success over a single pinhole (0.57 versus 0.31)** across tasks, while warning that fisheye policies "can overfit more easily to simple scenes" and recommending diverse backgrounds and Random Scale Augmentation (Rethinking Camera Choice, 2026). A plain A4 sheet is exactly such a simple scene (inference).

For your arm (inference, and subject to the team rule of testing before any physical change): moving the camera to the hand so it turns with the pusher is the one configuration in which a yaw action is clean. The plus or minus 145 degree hand-joint range plus the 45-degree visibility half-angle covers 190 degrees to each side, so a hairpin is reachable in principle, but a full 360-degree loop is not. With a hand camera the policy can output step directions in the camera's own frame plus a "turn camera left or right" class, which turns the Ned2 into the geometric equivalent of a mobile line follower, where a turn brings the line back into view and the entire lane-following literature applies directly. The demonstrator then simply keeps the line centred, so no VR headset or second teleoperation axis is needed. This is the most promising route to hairpins, but it is a hardware change: measure first whether a hand mount keeps the pusher vertical, whether the puck remains at the bottom of the frame, and what the turn classes do to episode length, and get the professor's agreement before committing.

Self-crossing loops are an ambiguity problem on top of the visibility problem

A crossing seen 7 cm ahead shows two valid continuations. No imitation-learning line follower found in the notes handles this without an explicit command input (Duckietown intersections) or memory; the Learning-to-Drive and Duckietown papers aggregate over whole maps without crossing-specific results (L■rincz et al. 2022). The demonstrated rule "continue straight through a crossing" removes the ambiguity from the data (Belkhale's consistency finding), a classification head keeps both branches as two probable classes rather than averaging them, and a heading-continuity preference is the obvious but untested tie-breaker (inference). Because loops also contain unobservable headings, treat the two-loop sheet as a stretch goal that depends on the camera decision above (inference).

4. Evaluate on the robot, per sheet, with counts and intervals

Offline accuracy will mislead you. The correlation between offline steering error and online success was only **0.39** with a centre camera and no action noise, rising to 0.77 only when validating on recovery-style three-camera data (Codevilla et al. 2018). robomimic found that choosing the checkpoint with the lowest validation loss gave policies **10 to 100 percent worse** than the best checkpoint (robomimic). A LeRobot post-mortem on a real arm reports a model that "kept grasping at a memorized position" despite low training loss (lerobot-project). Grade every model by closed-loop runs on the robot, and select checkpoints the same way.

The field's norms for real-robot evaluation are set out by Kress-Gazit et al. (2024): a "clear, detailed, and unambiguous definition of success" written in advance; report "the number of evaluations performed across each condition and not just percentages"; document initial conditions with overlays or markings; run all policies in one session because lighting and environment drift; state whether conditions are in- or out-of-distribution; record sub-goals per rollout; and tabulate failure categories. Their worked example shows that **13 of 20 versus 14 of 20 cannot be distinguished** under a uniform-prior binomial model, and that one policy fell from 13 of 20 to 0 of 8 after a lab reorganisation (Kress-Gazit et al.). Adding trials after seeing results and re-testing "amounts to p-hacking" (STEP). Trial counts in the literature are 20 to 25 per condition: ACT used 25 real evaluations per task with the object randomised along a 15 cm line and reported per-stage success (ACT, RSS 2023); Diffusion Policy used 20 real Push-T trials with success defined as final IoU above the worst demonstration's IoU, and reported duration alongside success (Diffusion Policy). Line-following work adds lateral deviation and distance: the AI Driving Olympics scored travelled distance, survival time, median lateral deviation and infractions, and called 20 episodes on one map statistically insufficient (Paull et al. 2019); a reinforcement-learning line

follower ran 20 trials per path (SAC-PID); an educational follower reports mean lateral error with a 95 percent confidence interval over 50 runs per condition (LineMaster Pro).

A protocol that matches these norms and your budget (inference): pre-register success as "puck centre within d_goal of the line end and never more than d_lat from the line centre at any step", fixing d_goal and d_lat from the demonstration set the way Diffusion Policy fixed its IoU threshold, for example the worst successful demonstration's maximum deviation (about 1 to 1.5 cm, half a puck, is the tolerance used in the pushing papers). Report per trial the completion fraction (line length covered before failure divided by total), steps until failure, mean and maximum lateral deviation of the puck centre measured from a top-down phone photo with the sheet corners as a homography reference, steps taken versus the demonstrator's steps, and a failure category (drifted off line, stuck, wrong branch at crossing, stopped early, step budget exceeded). Use a fixed step budget of about 1.5 times the demonstrated length as the cut-off. Run **20 trials per sheet** with marked, reproducible puck start positions, interleave conditions, and keep everything in one session with the same lamps. Report k of n with a binomial interval, never a bare percentage. Define "unseen" before training: a held-out sheet whose curvature and turn count lie inside the training range is an in-distribution generalisation test, and a tighter hairpin than any training sheet is out-of-distribution and should be labelled as such. With 25 to 40 s per episode plus resets, 20 trials is about 20 to 30 minutes of robot time per sheet, so one such evaluation per sheet is affordable; comparisons between model variants should be large-effect-only or use more trials.

Report results per path type, not as one number (inference). Straight lines and gentle arcs will likely reach high success from a few dozen episodes (robomimic's 20-percent-data result on easy tasks); the hairpin and the crossings are the Square and Transport of this task and will show where the data or the camera ran out.

5. Recommended plan: baseline, fallbacks, and open risks

The baseline

The baseline is a single-frame discrete-step classifier trained on keyboard-stepped demonstrations, then hardened with human-gated corrections (inference drawn from sections 1 to 3). Input: the 640x480 wrist frame downscaled to about 160x120, fixed-cropped to remove the rod and anything outside the sheet, randomly cropped to about 144x108 at training time, with brightness and contrast jitter of 0.9 to 1.1. Network: ImageNet-pretrained ResNet-18 fine-tuned end to end, or Hansen's 5-layer convolutional network from scratch, with a small head; learning rate 1e-4, Adam, batch 32 to 64, cross-entropy with class weights that oversample turn and hairpin frames. Output: six to nine classes (forward, forward-left, forward-right, left, right, backward, reposition moves, stop). Training on roughly 10,000 images of this size is minutes per epoch on a modern laptop CPU, so the full record-train-test-correct loop fits a weekly cycle (inference). Inference is tens of milliseconds, negligible against the one-second move (inference). Deployment loop: capture after the move returns, flush the stream, predict, execute the blocking 1 cm step, repeat until stop or step budget.

Ned2-specific facts to build around

The Niryo TCP server is documented as single-client and strictly sequential: "The server answers only after the command is done, so it can't deal with multiple commands at the same time" (Niryo TCP server docs, v3.2.0, page excerpt), and pyniryo sets its socket to blocking with no read timeout (pyniryo tcp_client.py). Your one step per second is therefore by design, not a bug, and image reads share the same serialised connection (inference). `jog_pose` skips motion planning and is the one call likely to shorten the round trip, but it is deprecated since 1.2.0 and has no published rate, so time it empirically before relying on it (inference). The only working Ned2 imitation-learning stack on record, the Niryo x Hugging Face LeRobot integration, needs a second Ned2 as leader, SSH changes, and killing the ROS stack, and its pull request was closed unmerged in August 2026 (LeRobot PR #1778; Niryo ned2.mdx); it is not a route for one robot and two beginners, and you should expect to be the first documented group doing BC through pyniryo on a Ned2 (inference). Practical checklist items: calibrate after every reboot because the Ned2 uses relative encoders and misreads near end-stops (Niryo ned2.mdx); use Ethernet because the Vision Set over Wi-Fi "may overload it, generating some latency" (Niryo Vision Set troubleshooting, page excerpt); and reproduce the sheet placement with markings, since repeatability of plus or minus 0.5 mm (robots.ros.org Ned2) is not the limiting factor but sheet position is.

Schedule to 10 January 2027

Weeks (2026-27)	Work	Done when
26 Oct to 8 Nov	Build the capture-then-move stepping script with keyboard control, stale-frame flush and full logging; measure the camera window with a ruler and `get_camera_intrinsics`; time `move` versus `jog_pose`; compute the observable fraction of each sheet; write the stepping rule; ask the professor the two questions (test-sheet family, hand-mounted camera)	20 clean trial episodes recorded and replayed; written rule; professor's answers
9 Nov to 22 Nov	Record 100 to 150 episodes over the six sheets plus 10 to 15 extra printed paths, with offset starts and oversampled curves; train the baseline; evaluate 20 trials on two seen sheets	First closed-loop numbers per path type

Weeks (2026-27)	Work	Done when
23 Nov to 6 Dec	Two human-gated correction rounds of 20 to 40 episodes each; retrain; first held-out-sheet evaluation	Held-out straight and arc sheets at target success
7 Dec to 20 Dec	Hairpin work: consistent repositioning demonstrations, lookahead labels, then a 10-frame GRU with past-action auxiliary loss only if needed; or the hand-camera route if approved and tested	Decision on hairpin and loop scope, with numbers
21 Dec to 3 Jan	Freeze the model; final pre-registered evaluation on the unseen sheet and all six sheets in one session; failure tables	All evaluation data collected
4 Jan to 10 Jan	Write-up, figures, code clean-up	Submission

Fallback ladder

If the baseline drifts on seen sheets (inference, ordered by cost): check timing and calibration before collecting more data, because a consistent directional miss in the closest forum analogue pointed to timing and camera placement rather than data quantity (HF forum thread); then add a correction round; then add DART-style perturbations and shift-and-relabel augmentation. If it fails only at hairpins: oversample hairpin frames, add lookahead labels, then the recurrent variant with copycat mitigation, then a 3-model ensemble (the 2026 chunking analysis credits part of chunking's benefit to implicit ensembling (Why chunking helps)). If a university GPU materialises and the professor values a modern architecture: train ACT through LeRobot on the same dataset as a comparison, with chunk size 2 to 3, and report it against the baseline rather than replacing it. If the camera cannot be changed and the test sheet contains unobservable stretches: report the measured observable fraction, the per-segment results, and the limitation honestly; a well-evaluated partial result with a clear cause is a legitimate course outcome.

Open risks

The dominant risk is the unobservable headings described in section 3; it is a hardware and scope question that no learning method resolves, and it needs the professor's input in the first two weeks. The second is compounding drift over 25 to 40 sequential decisions, which the recovery data and correction rounds address and which is why they are not optional. The third is lighting and session shift: a 2026 lighting study reports significant success drops under 50 percent shifts in lighting colour or direction (RoboLight, figures from a search excerpt and to be confirmed in the paper), and Kress-Gazit's 13 of 20 to 0 of 8 collapse after a lab reorganisation is the same lesson; the final evaluation must be in the same room, with the same lamps, at the same time of day, and pyniryo exposes no exposure control, so brightness jitter in training is the only software mitigation. The fourth is operator inconsistency between two students, controlled by the written rule and by discarding reversals. The fifth is checkpoint and model selection by validation accuracy, which the evidence says is unreliable. The sixth is interpretation: human-gated correction rounds involve the robot executing its own accepted steps during collection, and a scripted teacher would not be "pushing the puck ourselves"; both should be described to the professor rather than assumed acceptable. The seventh is time: a 3-week Christmas window sits in the middle of the hairpin work, and the schedule above assumes the robot stays available. Finally, every compute and resolution figure for your laptop is an extrapolation, and no source benchmarks CPU training of a small BC policy on about 10,000 wrist-camera frames, so measure your own epoch time in week one.

Conclusion

The literature's clearest message for this project is that the hard problems are not in the model. Across robomimic, the driving analogues and the pushing papers, three things separated working policies from failed ones: random-crop augmentation, recovery data in the training set, and evaluation in closed loop on the robot. A small classifier over 1 cm steps inherits the theoretical advantages that 2025 and 2026 papers attribute to quantised, non-smooth policies, and it trains in minutes on a laptop, which turns the whole project into a weekly loop of record, train, test and correct. That loop, not the choice between ACT and a CNN, is what the two students should optimise.

What the literature cannot do is give the camera a view it does not have. Every analogue that handled sharp turns either had a camera that turned with the vehicle, a wide lens, hand-coded search behaviour, or a second viewpoint, and the only one that quantified the narrow field of view found it to be the largest cause of failure. Measuring the observable fraction of each sheet, and settling with the professor whether the camera may move to the hand, is therefore the first task, because it decides the scope of everything after it. If the answer is no, the honest deliverable is a strong result on observable paths with a measured, explained limit; if yes, the Ned2 becomes a mobile line follower in geometry, and the oldest and best-tested recipe in visual imitation applies to the hairpins too.